



Relay402

— Prepared by Chris

Chris

ra3964@nyu.edu

01 — Project Overview

Relay402 is a Solana-based agent registry and pay-per-request escrow system. It associates service agents with HTTP endpoints, metadata, operators, payout accounts, request prices, and aggregate client feedback, while processing payments in a configured classic SPL token.

02 — Scope

Name	Description
relay402	Solana program relay402 under audit.

03 — Findings

Severity	Count
CRITICAL	0
HIGH	1
MEDIUM	2
LOW	4
INFO	1

04 — Vulnerabilities

ID	Severity	Description
SP-REL-ADV-00	HIGH	Ownership acceptance leaves the former operator and payout active, allowing post-transfer payments to be diverted.
SP-REL-ADV-01	MEDIUM	Zero-valued protocol fees let agents recycle payments and fabricate positive feedback without losing USDC.
SP-REL-ADV-02	MEDIUM	Agent closure ignores unrated settled receipts, allowing operators to suppress feedback and reset reputation.
SP-REL-ADV-03	LOW	Expired receipts can be refunded only by their clients, allowing withheld refunds to permanently block agent closure.
SP-REL-ADV-04	LOW	Agent reputation persists across ownership and service changes, allowing unrelated operators to inherit trusted ratings.
SP-REL-ADV-05	LOW	Ceiling-rounded fees exceed the 10% cap on small payments and can divert the entire amount from an agent.
SP-REL-ADV-06	LOW	Feedback checks only the current agent owner, allowing ownership rotation to legitimize self-issued ratings.
SP-REL-ADV-07	INFO	URI validators accept unsupported schemes, allowing unusable addresses to be persisted in Agent accounts.

Ownership Transfer Retains Prior Settlement Authority

HIGH

SP-

REL-ADV-00

Description

During `accept_agent_handler`, only `owner` and `pending_owner` are updated. The existing `operator`, `payout`, and `active` fields are retained, and no replacement payout is required from the new owner. Because `settle_payment_handler` authorizes settlement through `has_one = operator` and routes proceeds through `has_one = payout`, ownership is not coupled to settlement authority.

The former owner can therefore settle receipts that were pending during the transfer, as well as new receipts created before the new owner rotates both fields. Those rotation transactions may also be front-run because the agent remains active and no transfer-time freeze or minimum delay is enforced. Settlement requires no proof of delivery, while clients cannot refund pending receipts before expiry.

For a 10 USDC agent processing 100 requests/day with a 9.75 USDC net payment, 975 USDC/day can be transferred to the former owner while the new owner remains recorded on-chain and `settled_count` increases normally.

```
// programs/relay402/src/instructions/agent.rs
pub fn accept_agent_handler(ctx: Context<AcceptAgent>) -> Result<()> {
    let agent = &mut ctx.accounts.agent;
    let old_owner = agent.owner;
    agent.owner = ctx.accounts.new_owner.key();
    agent.pending_owner = None;

    // Operator and payout are NOT changed here. The new owner is expected to
    // rotate them with update_agent / set_payout. See README trust model.
    // ...
}
```

Remediation

Extend `AcceptAgent` to require a payout account owned by `new_owner` and distinct from `config.vault`. In `accept_agent_handler`, atomically set `operator` to `new_owner`, assign the supplied `payout`, and set `agent.active = false` until the new owner explicitly reactivates the agent.

Zero Protocol Fees Enable Reputation Forgery

MEDIUM

SP-REL-

ADV-01

Description

The protocol fee is constrained only by `MAX_FEE_BPS`, with no nonzero minimum enforced during configuration initialization. A `fee_bps` value of `0` is therefore accepted and stored; the same value can also be assigned through `update_config_handler`. Because each receipt snapshots this value, `compute_fee` returns zero during settlement and the entire payment amount is transferred to the agent-controlled payout account.

```
// programs/relay402/src/instructions/admin.rs
pub fn initialize_config_handler(
    ctx: Context<InitializeConfig>,
    fee_bps: u16,
    min_price: u64,
) -> Result<()> {
    require!(fee_bps ≤ MAX_FEE_BPS, Relay402Error::FeeTooHigh);
    require!(min_price > 0, Relay402Error::InvalidMinPrice);
    // ...
}
```

Payments can be created from a secondary wallet, settled to the agent's payout account, and followed by 5-star feedback. The owner-only self-feedback restriction is bypassed because the secondary wallet is treated as an independent client. By repeating the process with distinct nonces, the same USDC can be recycled at no protocol cost, excluding Solana transaction fees. Consequently, `settled_count`, `feedback_count`, and `score_sum` can be increased arbitrarily on-chain without any USDC loss.

Remediation

Add `require!(fee_bps > 0, Relay402Error::InvalidMinFee);` to both `initialize_config_handler` and the `fee_bps` update branch in `update_config_handler`. Reject all configuration changes that would set the protocol fee to zero.

Agent Closure Invalidates Unrated Settled Receipts

MEDIUM

SP-REL-

ADV-02

Description

`close_agent` permits an inactive agent to be closed whenever `pending_receipts` is zero. However, this counter tracks only `Pending` receipts: it is incremented by `create_payment_handler` and decremented by both `settle_payment_handler` and `refund_payment_handler`. No corresponding counter tracks `Settled` receipts awaiting feedback.

A settled receipt remains the client's authorization to call `submit_feedback_handler`, which requires the associated Agent PDA. Once that PDA is closed, feedback submission fails with `AccountNotInitialized`. Because `next_agent_id` increases monotonically, the closed identity cannot be restored under the same ID.

Consequently, payments may be settled, the agent may be deactivated and closed before negative feedback is submitted, and the same operator may register a new agent with the same `endpoint` or `metadata_uri`. On-chain, pending reviews become permanently unpostable while the replacement agent starts with `score_sum = 0` and `feedback_count = 0`.

```
// programs/relay402/src/instructions/agent.rs
pub struct CloseAgent<'info> {
    // ...
    #[account(
        // ...
        constraint = !agent.active && agent.pending_receipts == 0
        @ Relay402Error::AgentNotClosable,
        close = owner
    )]
    pub agent: Account<'info, Agent>,
}
```

Remediation

Add an `open_receipts: u64` or `unrated_settled: u64` field to `Agent`, increment it in `create_payment_handler`, and decrement it in `refund_payment_handler`, `submit_feedback_handler`, and `close_receipt_handler` while leaving it unchanged during settlement. Require this counter to be zero in `CloseAgent`, and authenticate the agent in `CloseReceipt` through `has_one = agent` or seed verification before decrementing it.

Client-Only Refunds Permanently Block Agent Closure

LOW

SP-

REL-ADV-03

Description

Each payment increments `agent.pending_receipts`, while the counter is decremented only through settlement or refund. Settlement is permanently disabled once

`now ≥ receipt.expires_at` because `settle_payment_handler` requires `now < receipt.expires_at`.

After expiration, `refund_payment_handler` is the only remaining resolution path. However, `RefundPayment` requires the original `client` as a signer and binds the receipt to `client.key()`. No permissionless `expire_receipt` or `force_refund` instruction is provided, so an expired receipt cannot be resolved when its client refuses to participate.

A minimum-price payment can therefore be created and deliberately left unrefunded after expiration. Because `close_agent` requires `!agent.active && agent.pending_receipts == 0`, the owner is permanently prevented from closing the Agent PDA, approximately `0.005 SOL` of agent rent remains unrecoverable, and `receipt.amount` remains locked in the shared `vault` on-chain.

```
// programs/relay402/src/instructions/payment.rs
#[derive(Accounts)]
pub struct RefundPayment<'info> {
    #[account(mut)]
    pub client: Signer<'info>;
    // ...
    #[account(
        mut,
        // ...
        has_one = client @ Relay402Error::ReceiptClientMismatch,
        close = client
    )]
    pub receipt: Account<'info, Receipt>;
    // ...
}
```

Remediation

Add a permissionless `expire_receipt` instruction that refunds `receipt.amount` to a token account constrained to `receipt.client`, decrements `agent.pending_receipts`, and closes the expired receipt. Alternatively, change `client` in `RefundPayment` to a `SystemAccount` and strictly derive or validate the client's destination token account without requiring the client's signature.

Reputation Persists Across Agent Identity Changes

LOW

SP-REL-

ADV-04

Description

The reputation counters `settled_count`, `feedback_count`, and `score_sum` are stored directly on the Agent PDA without being bound to its current owner, endpoint, metadata, or price. Consequently, historical ratings are presented as applying to the agent's current service even after its operational identity has changed.

Ownership can be transferred through `accept_agent_handler` without resetting these counters. Furthermore, `update_agent_handler` permits the endpoint, `metadata_uri`, `metadata_hash`, and price to be replaced while retaining the existing reputation.

```
// programs/relay402/src/instructions/agent.rs
pub fn update_agent_handler(ctx: Context<UpdateAgent>, args: UpdateAgentArgs) → Result<()> {
    // ...
    let agent = &mut ctx.accounts.agent;

    if let Some(endpoint) = args.endpoint {
        validate_endpoint(&endpoint)?;
        agent.endpoint = endpoint;
    }
    // ...
    if let Some(hash) = args.metadata_hash {
        agent.metadata_hash = hash;
    }
    // ...
}
```

```
// programs/relay402/src/instructions/agent.rs
pub fn accept_agent_handler(ctx: Context<AcceptAgent>) → Result<()> {
    let agent = &mut ctx.accounts.agent;
    let old_owner = agent.owner;
    agent.owner = ctx.accounts.new_owner.key();
    agent.pending_owner = None;

    // ...
}
```

A highly rated Agent PDA can therefore be sold or repointed to an unrelated service at a higher price while preserving its trusted average of `score_sum / feedback_count`. Because `settle_payment` requires no delivery proof, a client relying on that inherited rating can have an on-chain payment of up to the replacement `price` settled for a service that was never reviewed.

Remediation

Reset `settled_count` , `feedback_count` , and `score_sum` when ownership is accepted. Reset or version the reputation whenever `endpoint` , `metadata_uri` , `metadata_hash` , or `price` is materially changed so historical feedback cannot be attributed to a different service.

Ceiling-Rounded Fees Exceed the Configured Rate Cap

LOW

SP-

REL-ADV-05

Description

The nominal protocol fee is restricted to `MAX_FEE_BPS = 1_000`, but `compute_fee` rounds fractional fees upward by adding `BPS_DENOMINATOR - 1` before division. For any positive `fee_bps` where `amount * fee_bps < BPS_DENOMINATOR`, the calculated fee is raised to one base unit. The effective rate therefore becomes `1 / amount` and may substantially exceed both the configured rate and the advertised 10 % maximum.

```
// programs/relay402/src/utils.rs
pub fn compute_fee(amount: u64, fee_bps: u16) → Result<u64> {
    let fee = (amount as u128)
        .checked_mul(fee_bps as u128)
        .and_then(|v| v.checked_add((BPS_DENOMINATOR - 1) as u128))
        .and_then(|v| v.checked_div(BPS_DENOMINATOR as u128))
        .ok_or(Relay402Error::MathOverflow)?;
    let fee = u64::try_from(fee).map_err(|_| Relay402Error::MathOverflow)?;
    require!(fee ≤ amount, Relay402Error::MathOverflow);
    Ok(fee)
}
```

Only a positive `min_price` is required, and no post-calculation effective-rate or positive-net check is performed in `settle_payment_handler`. For example, an `amount` of 1 with any positive permitted fee produces `fee = 1` and `net = 0`. When such a receipt is settled, the entire escrowed payment is transferred to the treasury while the agent's payout receives zero tokens.

Remediation

Remove the ceiling adjustment from `compute_fee` and calculate the fee using floor division of `amount * fee_bps` by `BPS_DENOMINATOR`. Add boundary tests confirming that the resulting fee never exceeds `amount * MAX_FEE_BPS / BPS_DENOMINATOR`.

Description

Self-feedback is rejected by `submit_feedback_handler` only when the client matches the agent's current owner. The owner associated with the agent when the payment was created or settled is not recorded. Because `transfer_agent_handler` and `accept_agent_handler` replace `agent.owner` without retaining ownership history, eligibility for feedback can be changed after settlement.

```
// programs/relay402/src/instructions/payment.rs
pub fn submit_feedback_handler(ctx: Context<SubmitFeedback>, score: u8) ->
Result<> {
    // ...
    // Blocks the trivial case only. A second wallet can still rate, but every
    // score costs a real settled payment (price ≥ min_price, plus fee).
    require!(
        ctx.accounts.client.key() ≠ agent.owner,
        Relay402Error::SelfFeedback
    );
    // ...
}
```

A payment may therefore be created and settled from an owner-controlled wallet, after which ownership may be transferred to a second wallet. Feedback from the original wallet is then accepted, and ownership may subsequently be transferred back. Consequently, `feedback_count` and `score_sum` can be increased on-chain by self-issued feedback while the original owner ultimately retains the agent.

Remediation

Add an owner snapshot to `Receipt` when the payment is created or settled. Reject feedback when `receipt.client` matches that snapshot, regardless of subsequent ownership transfers.

Description

Agent endpoints are documented as HTTPS URLs, while metadata locations are documented as HTTPS or IPFS URLs. However, `validate_endpoint` and `validate_metadata_uri` only enforce that their inputs are non-empty and do not exceed 200 bytes. No URI parsing or scheme allowlist is applied, so values using unsupported schemes such as HTTP or FILE satisfy validation.

```
// programs/relay402/src/utils.rs
pub fn validate_endpoint(endpoint: &str) → Result<> {
    require!(
        !endpoint.is_empty() && endpoint.len() ≤ MAX_ENDPOINT_LEN,
        Relay402Error::InvalidEndpoint
    );
    Ok(())
}
```

Both `register_agent_handler` and `update_agent_handler` rely on these validators before assigning the supplied values to `Agent.endpoint` and `Agent.metadata_uri`. Consequently, clients may attempt to resolve unsupported or unusable locations and fail to communicate with registered agents. Unsupported URI schemes are therefore accepted and persisted on-chain in Agent accounts.

Remediation

Require `endpoint` values to use `https://` and `metadata_uri` values to use either `https://` or `ipfs://`. Reject malformed URIs and all schemes outside these explicit allowlists.